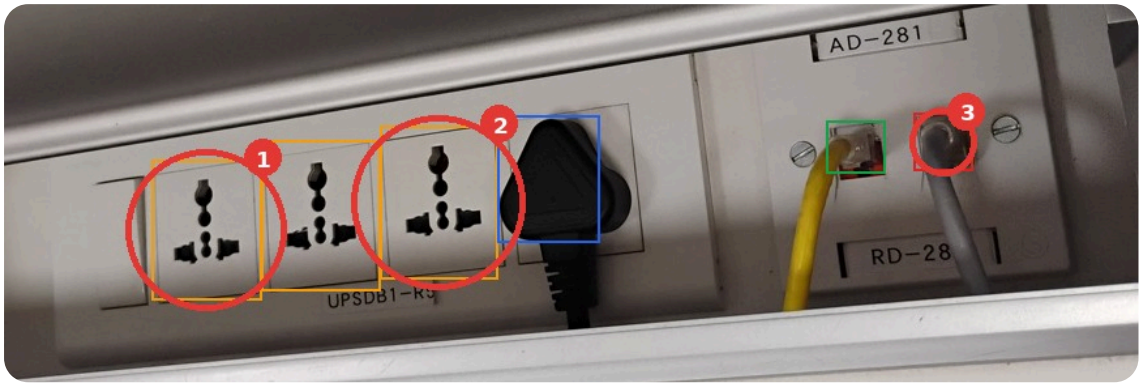
On-Device AI Install Checker — YOLO engine

Same app, same checklist — a second, much smaller AI engine: YOLO object detection + fixed rules.



- Model: **YOLO26n** (Ultralytics), trained on our 9 photos
- **5 MB** instead of 3.86 GB — downloads in seconds
- Finds **14 / 14** problems on the demo photos (Gemma: 12 / 14)
- One model, **iOS (Core ML)** and **Android (TFLite)**
- Fully offline · switch engines in the app's *About* screen

1 · Why a second engine?

	YOLO26n + rules	**Gemma 4 E2B (LoRA)**
What the AI does	finds each socket / port and its state	reads the whole photo and writes the answer
Who writes the explanation	fixed rules → always the same wording	the model itself
Model size on phone	**4.9 MB** (Core ML) · 9.9 MB (TFLite)	3.86 GB
Issues found on the 9 demo photos	**14 / 14**, 0 false alarms	12 / 14
Training	~50 min on a laptop CPU	~2.5 h on MacBook GPU + 10 min convert
Android	ready-made `.tflite`	same `.litertlm` (LiteRT-LM)
Best for	fixed checklists, speed, accuracy of the circle	free-form questions, new kinds of problems

Both engines output the same JSON, `{"status","errors":[{x,y,r,desc}]}`, so the app draws the circles the same way.

2 · How does the AI "see" a photo?

The AI does one job: it finds every socket and port, and names what is in it. A short checklist then decides pass or fail.



① Photo
the technician takes a picture



② AI finds & names
"empty, empty, empty, plug, LAN, extra cable"

FAIL · 3 issues
- × socket 1: adapter missing
- × socket 3: plug missing
- × grey cable plugged in
- ✓ LAN cable OK

③ Checklist decides
compare with the standard setup

- This kind of AI is called **object detection**. It is the same idea a car uses to spot pedestrians, just trained on our 6 things.
- To teach it, we show it photos where a person has already drawn and named every box. The next 5 slides are those steps.

3 · Training step 1 — Collect photos

1 · Collect photos | 2 · Draw boxes | 3 · Make variations | 4 · Practice (train) | 5 · Test & package

Take real photos of the desk box, both correct and wrong installs.



- **9 photos** for the demo: 4 correct, 5 with mistakes (14 mistakes in total).
- Every kind of mistake needs examples: a missing adapter, a missing plug, a wrong cable.
- Photos are straightened and resized the same way the app will do it on the phone.
- **For production:** 30–50 photos per situation, from different rooms, lighting and angles.

4 · Training step 2 — Draw boxes and name them

1 · Collect photos | **2 · Draw boxes** | 3 · Make variations | 4 · Practice (train) | 5 · Test & package

A person draws a box around every socket and port and picks its name. This is the "answer key" the AI learns from.



Name	Meaning
adapter	power adapter plugged in
plug	power cord plugged in
empty	empty power socket
LAN	yellow LAN cable
extra	any other cable
port	empty data port

- **53 boxes** on 9 photos. Tools like Label Studio or Roboflow make this point-and-click.
- We label *what we see*, not "right" or "wrong". If the standard setup changes, the labels stay the same.

5 · Training step 3 — Turn 9 photos into thousands

1 · Collect photos | 2 · Draw boxes | **3 · Make variations** | 4 · Practice (train) | 5 · Test & package

Each time the AI practices, the photos are shuffled, tilted, zoomed and stitched together, so it never sees the exact same picture twice.

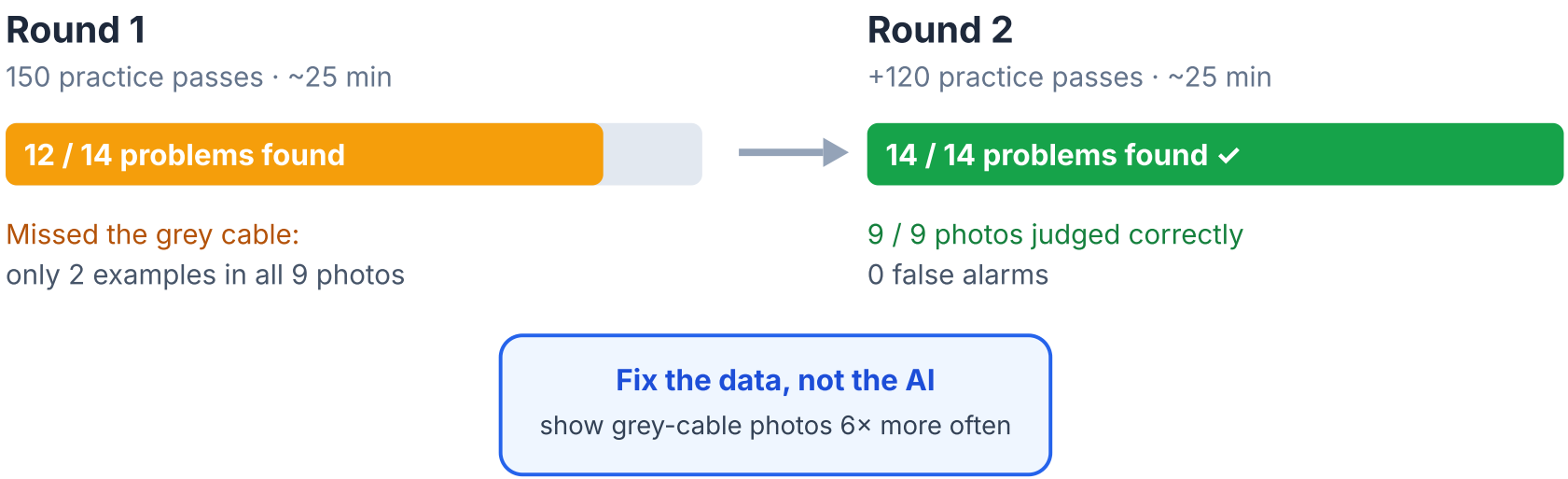


- Like showing a new employee the same desk from different angles and in different light.
- The boxes move with the picture, so the answer key stays correct.
- We keep the **colors** almost unchanged, because yellow vs. grey cable is exactly what the AI must notice.
- Rare mistakes (the grey cable) are shown more often so the AI does not ignore them.

6 · Training step 4 — Practice, check, repeat

1 · Collect photos | 2 · Draw boxes | 3 · Make variations | **4 · Practice (train)** | 5 · Test & package

The AI guesses the boxes, compares with the answer key, and corrects itself, hundreds of times.

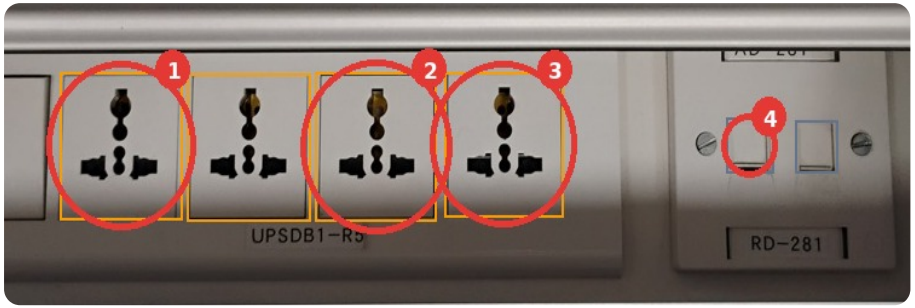


- We start from a model that already knows how to find everyday objects, and only teach it our 6 new names. That is why 9 photos are enough for a demo.
- Runs on a normal laptop: about **50 minutes** in total, no cloud and no special hardware.

7 · Training step 5 — Test and package for the phone

1 · Collect photos | 2 · Draw boxes | 3 · Make variations | 4 · Practice (train) | **5 · Test & package**

Check the AI + checklist against the 14 known mistakes, then shrink it into one small file the phone can run.



wrong install → red circles

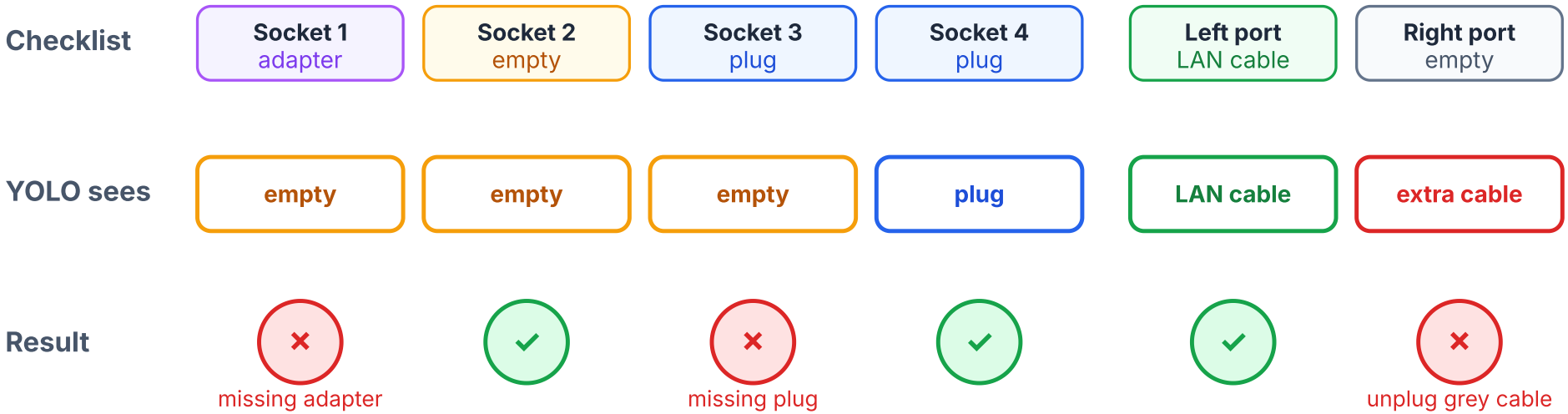


correct install → PASS

Test result	Packaged file
14 / 14 mistakes found · **9 / 9** photos judged correctly · 0 false alarms	**iPhone:** Core ML, 4.9 MB · **Android:** TFLite, 9.9 MB · uploaded to GitHub, the app downloads it in seconds

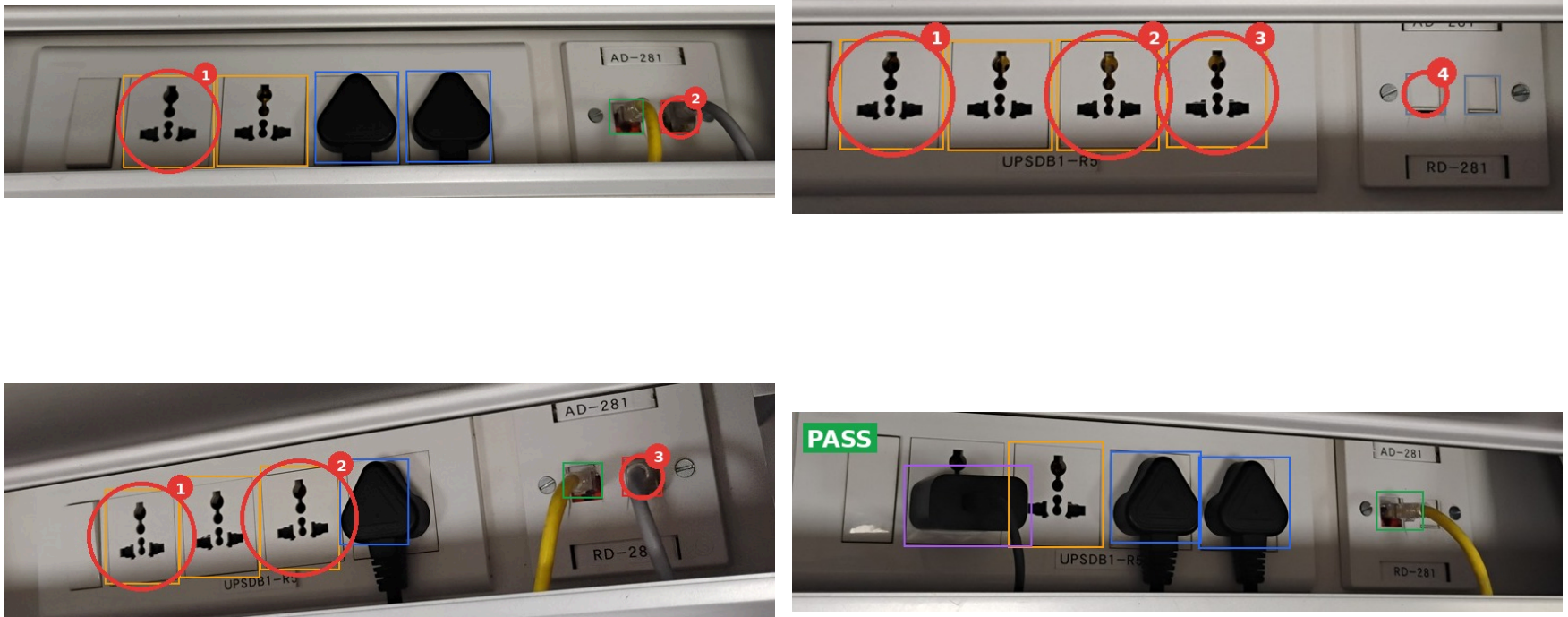
8 · From boxes to problems: the checklist rules

YOLO only says *what is in each slot*. A few lines of code compare that with the standard setup.



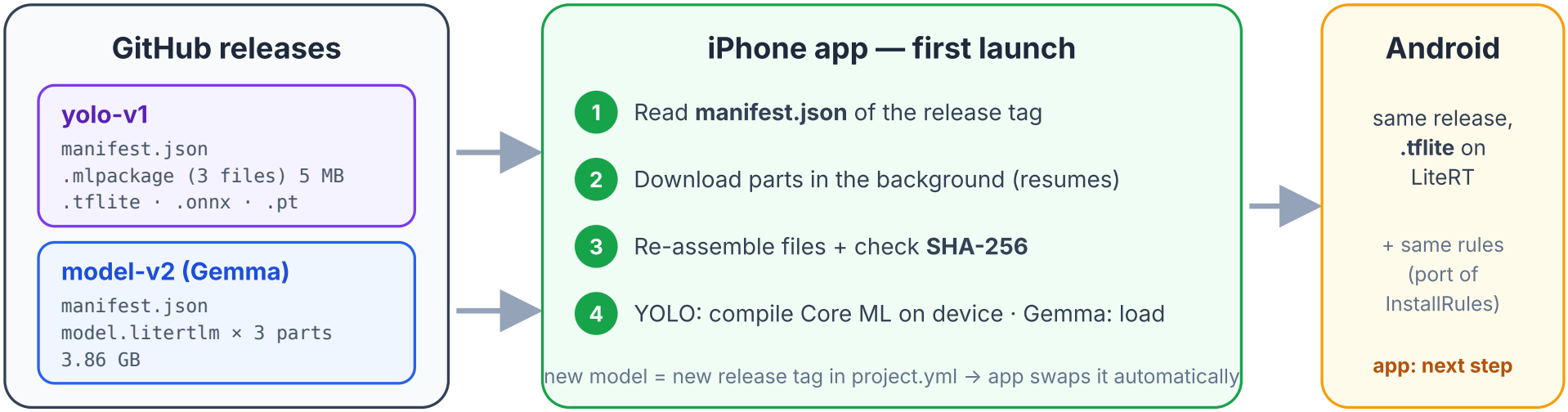
- Sockets are sorted **left → right** and must be: adapter · empty · plug · plug. The data ports need **one yellow LAN cable** and **no other cable**.
- Every mismatch becomes one red circle around that slot, plus a fixed fix-it sentence, e.g. *"Missing power plug. Plug a power cord into the third socket."*
- The rules live in `InstallRules.swift` (app) and `yolo/rules.py` (training repo). Changing the checklist means changing the rules, not retraining.

9 · Input → Output



Thin boxes = what YOLO detected · red circles = problems found by the rules · bottom-right: a correct install → PASS.

10 · Shipping the models: GitHub releases



- Models are **not** inside the app. Each model is a GitHub release with a `manifest.json` (files, sizes, SHA-256): `yolo-v1` and `model-v2`.
- A new model is a new release tag; changing the tag in `project.yml` makes every phone download it once.

11 · Numbers

	Result
PASS / FAIL correct	**9 / 9** photos
Problems found	**14 / 14**, 0 false alarms (Core ML is the same network as the tested PyTorch / ONNX / TFLite builds)
Detection quality (mAP50)	**0.98** on the demo photos
Model file	Core ML **4.9 MB** · TFLite **9.9 MB** · ONNX 9.9 MB
Training	2 rounds × ~25 min on a laptop CPU, 2.4 M parameters

Honest limits today: the same as the Gemma engine. There are only 9 photos of one desk, so the model has memorized this demo and won't generalize yet. YOLO also only knows the 6 states it was taught, and anything new needs a new class plus a rule. The Core ML build has not yet been timed on the iPhone.

12 · Next steps

- **Real data:** 30–50 photos per state from technicians (different rooms, lighting, angles), boxed with the same 6 classes, retrained with the same 4 steps (about an hour).
- **Measure on device:** time per photo and accuracy on iPhone 15 Pro with the Core ML build.
- **Android app:** load `NetworkChecker.tflite` with LiteRT and port the ~60 lines of `InstallRules`.
- **Combine engines (later):** YOLO for fast, exact circles + Gemma for free-form explanations of unusual problems.
- **License:** Ultralytics YOLO is AGPL-3.0, which is fine for this POC. A product needs an Ultralytics Enterprise License, or a switch to Apple Create ML (iOS only).